

Trace 追踪（可观测与链路排查）

三层观测模型、质量信号、调用链下钻与排查闭环 · 2026-07-12

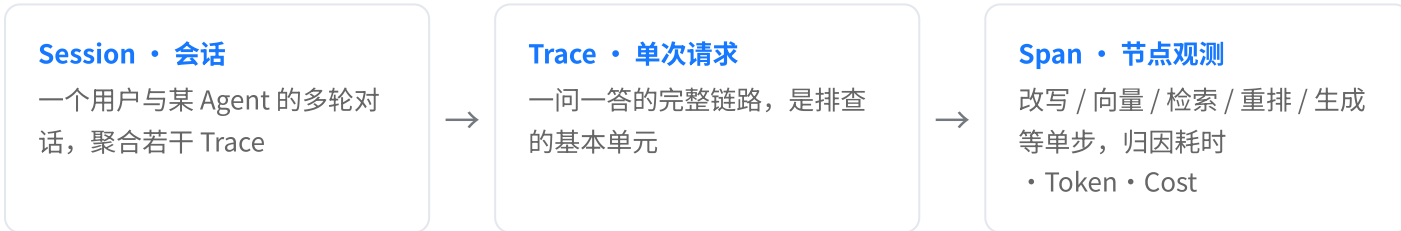
一、定位与用户

Trace 追踪是系统的**可观测层**——问答链路「配置 → 验证 → 上线 → 追踪」的最后一环，也是唯一回答「线上这次回答到底是怎么生成的、慢在哪、错在哪」的地方。它把每一次 C 端问答拆解成可回放、可下钻、可归因的调用链，既服务日常巡检（失败率 / 耗时是否异常），也服务定点排查（某条被点踩的回答，问题出在检索、Prompt 还是模型）。

使用者是运营 / 算法 / 研发同学。它不对 C 端用户可见——Session 详情里虽然还原了 C 端聊天窗口的样子，但每条回复下方的「溯源条」是只有运维能看到的观测入口。Trace 与「知识缺口 / Badcase」「评测集」「Prompt」三个模块强耦合：Trace 是问题的**发现与诊断起点**，其余模块是**沉淀与修复终点**。

二、三层观测模型

对齐 OpenTelemetry 的 GenAI 语义约定，采用「Session → Trace → Span」三层结构。分层的意义是让排查能在不同粒度间自由缩放：从「这个用户的整段对话」到「这一次请求」再到「请求里的某个节点」。



Span 采用 OTLP 风格模型：`start / dur` 为相对起点的偏移毫秒，`status` 取 OK / ERROR / WARN / SKIP，`kind` 标注节点类型，Token 与 Cost 归因到产生它的具体 span。底层按此结构落 ClickHouse，导出 JSON 时用 `gen_ai.usage.input_tokens`、`gen_ai.cost.cny` 等标准属性名，便于与外部可观测体系互通。

三、列表页：Trace / Session 双视图

列表页用一个分段控件切换两个视角——**Trace**（面向单次请求，做指标巡检与逐条排查）与 **Session · 会话**（面向完整对话，理解上下文与多轮问题）。二者共享同一批底层数据，只是聚合粒度不同。

1. Trace 视图

顶部四张概览卡先给出健康度全局判断：**采样 Trace 数**、**失败率**（含失败条数，异常时标红）、**P95 耗时**（含超时熔断请求，≥5s 标红）、**快捷排查**（全部 / 失败 / 慢请求 / 低分召回 一键筛选，把最常见的三类问题做成预设）。

筛选行支持：关键词（问题 / Trace ID）、Agent、状态、时间范围。列表列为 **Trace ID · 时间 · 用户问题(附质量信号标签) · Agent · 状态 · 总耗时 · Tokens**，点行进入详情。空态展示「没有符合条件的 Trace」。

2. Session 视图

列表列为 **Session ID · 用户 · Agent · 轮次 · 首轮问题 · 状态**。会话级状态由内部各轮聚合而来：任一轮失败即「含失败」（红），无失败但有兜底即「含兜底」（黄），否则「正常」（绿）——让人一眼看出「这段对话整体顺不顺」。

四、状态与质量信号

两套正交的标注体系：**状态**描述请求成没成，**质量信号**描述回答好不好——一条请求可以「成功」却携带多个质量信号（如成功返回但被用户点踩）。

状态	含义
成功	链路正常完成并返回回答
兜底	未命中知识，走兜底话术；链路本身正常
失败	某个 span 报错 / 超时熔断，未产出有效回答

质量信号在列表里作为问题行的内联小标签出现，并自动汇入 Badcase 问题池供人工分类分流：

低召回分

无引用

生成拒答

超时

用户点踩

连续追问

前四个来自系统自动判定（检索分、引用、模型返回、耗时），后两个来自用户行为（C 端点踩、同一话题反复追问），共同构成「哪些回答值得回头看」的信号面。

五、Session 详情：C 端还原 + 溯源下钻

Session 详情**1:1 还原该会话在 C 端的真实聊天窗口**（微信式绿色用户气泡、Agent 头像、在线状态、装饰性输入栏），让排查者先站在用户视角读懂对话，而不是一上来就陷进技术细节。

关键设计是每条 bot 回复气泡下方挂一条**溯源条**：状态标签 + Trace ID + 耗时 + 「链路 →」。它是观测层专属、C 端不可见，点击即从「用户看到的回答」下钻到「这次请求的调用链」。这条「体验 ⇄ 链路」的通道，是把抽象的 Trace 数据锚定到具体对话体验上的核心交互。

六、Trace 详情：调用链下钻

1. 头部元信息与操作

顶部展示用户问题与六项归因元信息：**Agent · 生成模型(含版本) · Prompt 版本 · 总耗时 · Tokens(入/出) · Cost**。右上角四个操作构成排查后的出口动作：**+ 加入评测集**（把这条真实问题沉淀为回归用例）、**U 重放**（带参跳评测复现）、**跳转 Prompt 版本 →**（定位到当次用的 Prompt 去改）、**{ } 复制 JSON**（导出 OTLP 结构给研发）。

2. 调用链：时间轴 / 树形双视图

左栏是调用链，提供两种视图：**时间轴**（瀑布图，按 span 的起止偏移排布，一眼看出耗时都花在哪一段、哪段串行哪段重叠）与**树形**（按父子层级展开，附节点类型标签、Token、Cost、耗时）。span 类型用颜色区分并附图例：

顶部固定一行 TRACE 根节点代表整条请求。**失败 Trace 默认自动定位到报错的 span**（标红），省去手动翻找——打开即见问题现场。SKIP 态的 span（如上游超时后被跳过的后处理）以灰色弱化呈现。

3. Span 详情面板

右栏随左侧选中的 span 联动，按节点类型展示不同内容——这是「按需展开、不同节点看不同东西」的设计：

- **错误框**（ERROR 时）：错误类型 + 详细信息，如「DeadlineExceeded · 上游模型网关 30,000ms 未返回，请求被熔断」；
- **基础 meta**：Session / User / 环境 · 版本 / Agent 等键值；
- **Scores 评估**（根节点）：整条 Trace 的评估打分卡；
- **检索命中分表**（检索/重排节点）：每个命中分块的**向量分** · **关键词分** · **Rerank 分** · **结果（命中/淘汰）**——Rerank ≥0.65 绿、否则红，直接看出召回质量与融合效果；
- **引用来源**：角标 ↔ 命中分块的对应关系，与 C 端回答里的角标一致；
- **输入 / 输出**：该 span 的实际入参与产出，输入带「已脱敏」标记（隐私合规）。

七、排查闭环

Trace 不是终点，而是「发现问题 → 定位根因 → 推动修复 → 回归验证」链条的起点。三条出口路径把诊断结果导向对应的修复模块：

出口	去向	用途
+ 加入评测集	评测集（如「退款场景回归集」）	把真实 badcase 固化为回归用例，防止改坏后复发
↻ 重放	效果评测 · 带参运行	用当次输入在新配置下复现，验证是否已修复
跳转 Prompt 版本	Prompt 详情 / Playground	定位到当次用的 Prompt 直接调优
质量信号自动汇入	Badcase 问题池	带信号的问题批量沉淀，按原因（知识缺失/检索/Prompt/幻觉…）人工分流

反向地，Agent 列表的「日志」入口、独立的 Trace 详情页的「用于测试」按钮，也都指向 / 来自 Trace，形成「配置 ⇄ 追踪」的双向可达。

八、边界条件与异常态

场景	页面处理
失败 Trace（span 报错/超时）	详情自动定位报错 span，红色错误框摊开类型与信息
超时熔断后的后续节点	标为 SKIP，灰色弱化，Tokens 显示「—」

场景	页面处理
兜底 Trace	状态黄标，调用链走兜底话术 Prompt，检索分普遍偏低
筛选无结果	列表空态提示 + 一键「重置」筛选
从 Agent「日志」进入	自动按该 Agent 名预置筛选，其余条件清空
输入/输出含隐私	输入区「已脱敏」标记，落库前处理敏感字段
采样与留存	概览标注「采样 Trace 数」，非全量；留存策略见后续

九、明确不做 / 后续方向

能力	当前取舍
实时告警 / 阈值订阅	当前为事后巡检，失败率 / P95 主动告警留待接入监控体系
自定义指标看板 / 下钻聚合	概览卡为固定三指标，多维分析暂由运行看板承担
采样率 / 留存周期配置 UI	当前采样策略后台固定，暴露为配置项待真实成本压力出现
跨 Trace 的根因聚类	当前逐条 + 质量信号分流，自动聚类同类问题为后续增强
脱敏规则自定义	当前默认脱敏，字段级规则配置留后续

整体判断：Trace 已把「三层观测 + 质量信号 + 调用链下钻 + 排查闭环」的核心链路做扎实，能支撑从「发现一条坏回答」到「推动一次修复」的完整路径。后续增强集中在**主动性（告警）、聚合分析（看板/聚类）与治理配置（采样/脱敏）**三处，均为在既有 OTLP 模型上的能力扩展，不改变三层结构与页面骨架。